PrivTI: Efficient End-to-End Privacy-Preserving Inference for Transformer-Based Models in MLaaS

Mingshun Luo, Haolei He, Wenti Yang, Meng Li, *Senior Member, IEEE*, Longfei Wu, *Member, IEEE*, Zijian Zhang, *Senior Member, IEEE*, and Zhitao Guan, *Member, IEEE*

***Abstract*—Machine Learning as a Service (MLaaS) offers convenient access to high-quality transformer-based model inference without local deployment, but it requires uploading sensitive data to the server, raising significant privacy concerns. Recent advances in homomorphic encryption (HE) and secure multi-party computation (MPC) have shown great potential for enabling privacy-preserving inference. However, existing schemes face challenges with transformer-based models due to massive matrix multiplications and complex non-linear functions. To address these challenges, we propose PrivTI, an efficient end-to-end privacy-preserving inference framework for transformer-based models. Specifically, we design an HE-based single-instruction multiple-data (SIMD) encoding scheme to perform matrix multiplication efficiently. To handle non-linear functions, we develop MPC-friendly protocols for core functions like GELU, Softmax, and LayerNorm, and further extend support to SiLU and RMSNorm. Moreover, PrivTI supports conversions between HE and MPC, enabling end-to-end privacy-preserving inference. Experimental results demonstrate that PrivTI achieves speedups 1.9–45.2× in matrix multiplication and 1.1–18.9× in non-linear function evaluation compared to baseline schemes IRON, BOLT, and NEXUS under the Local Area Network (LAN) setting. Additionally, PrivTI reduces communication overhead by 1.1–4898.7×. Moreover, end-to-end benchmarks show 3.3–9.8× improvements in computation speed and 1.6–6.2× reductions in communication overhead.**

***Index Terms*—Privacy-preserving machine learning, private transformer inference, secure multi-party computation, homomorphic encryption.**

I. Introduction

THE rapid growth of Machine Learning as a Service (MLaaS) allows users to easily access powerful transformer-based models and computing resources for inference without the need to build their own infrastructure [1]. It enables users to utilize cutting-edge model capabilities on demand and quickly integrate intelligent services into applications, such as multimedia analytics [2] and predictive maintenance in industry [3]. According to IMARC Group's forecast, the global MLaaS market is expected to grow from $9.6 billion in 2024 to $84.1 billion by 2033 [4].

With the accelerating adoption of MLaaS, transformer-based model inference increasingly involves sensitive data, such as user logs, dialogues, or medical images. This introduces significant privacy risks [5], making privacy protection a critical challenge for the adoption of MLaaS. Recent advances in homomorphic encryption (HE) and secure multi-party computation (MPC) offer promising solutions [6], [7], [8], [9], [10], [11], but their high computational and communication overhead often result in severe efficiency degradation. Moreover, these techniques require modifying the model architecture to support non-linear operations, resulting in accuracy degradation [12]. Consequently, realizing privacy-preserving transformer inference with minimal impact on computational and communication efficiency while maintaining comparable accuracy has become a key challenge in MLaaS.

Despite growing interest in privacy-preserving inference, existing schemes are largely designed for small-scale deep learning models [13], [14], [15], [16], [17], [18], especially convolutional neural networks (CNNs), and primarily rely on HE or MPC to protect data privacy during inference. Compared to CNNs, extending these schemes to transformer-based models is considerably more difficult due to large-scale matrix multiplications, complex non-linear functions, and deeply stacked layers that amplify both computational and communication overhead [19]. For example, with a sequence length of 128, BERT-base and GPT-2-base perform repeated head-wise projections of size $128\times768\times64$ across 12 layers, in addition to large feed-forward matrix multiplications. Besides, transformers introduce complex operators such as GELU, Softmax and LayerNorm, which involve costly operations like Gaussian error computation, exponentiation, division, and square roots [20].

In recent years, research has increasingly focused on privacy-preserving inference for transformer architectures, with an emphasis on optimizing two key components: matrix multiplication and non-linear function evaluation.

1) Matrix multiplication: One existing approach is two-party computation (2 PC) plus HE protocols (e.g., IRON [21] and

Received 30 September 2025; revised 19 February 2026; accepted 14 April 2026. Date of publication 17 April 2026; date of current version 9 October 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62372173, Grant 62372149, Grant U23A20303, and Grant 62572168 and in part by the Anhui Provincial Natural Science Foundation under Grant 2508085MF151. *(Corresponding author: Zhitao Guan.)*

Mingshun Luo, Haolei He, and Zhitao Guan are with the School of Control and Computer Engineering, North China Electric Power University, Beijing 102206, China (e-mail: lms1216110562@gmail.com; 1452567395@qq.com; guan@ncepu.edu.cn).

Wenti Yang is with Pengcheng Laboratory, Shenzhen 518066, China (e-mail: yangwt@pcl.ac.cn).

Meng Li is with the School of Computer Science and Information Engineering, Hefei University of Technology, Hefei 230601, China (e-mail: mengli@hfut.edu.cn).

Longfei Wu is with the Department of Mathematics Computer Science, Fayetteville State University, Fayetteville, NC 28301 USA (e-mail: lwu@uncfsu.edu).

Zijian Zhang is with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China (e-mail: zhangzijian@bit.edu.cn).

Digital Object Identifier 10.1109/TSC.2026.3685039

1939-1374 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on November 08,2026 at 04:12:15 UTC from IEEE Xplore. Restrictions apply.

BumbleBee [19]), which encrypts one party's shares and applies polynomial packing, but it still incurs substantial communication overhead. Alternatively, MPC-Dealer and three-party computation (3 PC) schemes (e.g., Privformer [22] and PUMA [23]) achieve better efficiency but depend on trusted third parties. More recent HE-based solutions such as BOLT [24] and NEXUS [25] improve throughput through single-instruction multiple-data (SIMD)-style encoding, enabling parallel computation by packing multiple data elements into SIMD slots, but still suffer from high latency caused by expensive ciphertext rotation operations.

2) Non-linear function evaluation: A common approach for approximating GELU is piecewise polynomial approximations. However, the large number of polynomial segments used in PUMA [23] and BumbleBee [19] requires additional interval comparisons, which significantly increases communication and computation overhead. Additionally, the piecewise polynomial approximations adopted by BOLT [24], PUMA [23], and BumbleBee [19] suffer from considerable approximation errors. For the exponential function in Softmax, schemes such as PUMA [23], BumbleBee [19], and NEXUS [25] rely on Taylor series expansions, with polynomial degrees as high as 64 to ensure sufficient accuracy, which further increases computational overhead. Moreover, research on privacy-preserving protocols for non-linear operators such as SiLU and RMSNorm in LLaMA models remains limited.

A. Our Approach

To address the above challenges, we propose PrivTI, an efficient end-to-end privacy-preserving inference framework for transformer-based models in MLaaS. It focuses on optimizing both matrix multiplications and non-linear function evaluations.

1) Matrix multiplication: We adopt HE to perform matrix multiplications and leverage SIMD-style encoding in HE to enable parallel operations across multiple data slots in a ciphertext. Specifically, we propose a pipeline that processes the original ciphertext according to specific rules to generate multiple independent extended ciphertexts. In this design, only a single round of parallel multiplication is required, and summing the results yields the final output of the matrix multiplication. Building upon this pipeline, we develop a matrix multiplication protocol with tailored SIMD-style encoding. When HE is applied, computations must be adapted to the properties of encrypted operations. Accordingly, we design distinct SIMD encodings for ciphertext–plaintext (Ct–Pt) and ciphertext–ciphertext (Ct–Ct) matrix multiplications. The protocol is carefully optimized to minimize the number of rotations, which further accelerates computation and reduces latency. In addition, compact packing is used to enhance data utilization, thereby reducing communication overhead.

2) Non-linear function evaluation: We adopt MPC to evaluate non-linear functions, leveraging polynomial approximation and lookup tables to construct MPC-friendly functions. For GELU, we approximate the function over large intermediate intervals with high-order polynomials to reduce the number of segments, and primarily use even-order terms to take advantage of the square protocol in MPC. For the exponential function in Softmax, we decompose the input into integer and fractional parts, approximating the fractional part over a small interval, while the integer part is efficiently handled using a compact lookup table. In addition, we design a privacy-preserving LayerNorm protocol based on Newton's iteration. Furthermore, we extend our protocols to support SiLU and RMSNorm, which are widely used in modern transformer architectures such as LLaMA.

B. Contributions

The key contributions of this paper are as follows:

- We propose PrivTI, an efficient end-to-end privacy-preserving inference framework for transformer-based models in MLaaS that combines HE-based matrix multiplication and MPC-based non-linear function evaluation to minimize computational and communication overhead while achieving accuracy comparable to plaintext inference.
- We design a novel HE-based matrix multiplication protocol with tailored SIMD-style encoding. It splits ciphertexts into multiple extended ciphertexts, performs parallel computation in a single multiplication round to accelerate computation, and applies compact packing to reduce communication overhead.
- We develop optimized MPC-friendly evaluation protocols for core non-linear functions (GELU, Softmax, LayerNorm), with extensions to support SiLU and RMSNorm. Moreover, the proposed approximation polynomials exhibit minimal error relative to the original functions, enabling efficient privacy-preserving inference while maintaining comparable accuracy.

C. Organization

The remainder of this paper is organized as follows. Section II reviews related work, and Section III presents the preliminaries. Section IV describes our efficient privacy-preserving transformer inference framework. Sections V and VI detail the proposed matrix multiplication protocol and non-linear function evaluation protocol, respectively. Section VII shows the end-to-end workflow. Experimental results are presented in Section VIII, followed by discussion in Section IX, and conclusions are drawn in Section X.

II. Related Work

A. HE-Based Privacy-Preserving Transformer Inference

Recent efforts have explored HE-based solutions for privacy-preserving transformer inference. THE-X [26] approximates GELU, Softmax, and LayerNorm with HE-friendly functions, enabling end-to-end encrypted inference. Building on this foundation, Rovida et al. [27] adopts CKKS with bootstrapping and precomputed LayerNorm statistics to lower circuit complexity. THOR [28] improves BERT-base inference through optimized matrix multiplication, compact packing, and iterative non-linear approximations, while Zimerman et al. [29] convert self-attention and LayerNorm into polynomial forms. To further

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on November 08,2026 at 04:12:15 UTC from IEEE Xplore. Restrictions apply.

improve efficiency, Powerformer [30] replaces Softmax and LayerNorm with lightweight functions and optimizing matrix multiplication. More recently, NEXUS [25] introduces ciphertext compression and slot folding to achieve non-interactive inference, and ENSI [31] further improves efficiency and scalability.

B. MPC-Based Scheme

MPC-based schemes primarily focus on optimizing costly non-linear functions. SecureGPT [32] provides a comprehensive set of building blocks, including conversion of multiplicative shares to additive shares, truncation, division, and protocols for Softmax and GELU, covering all transformer layers. SecureTLM [33] employs hybrid secret sharing to preserve accuracy without approximation loss. SIGMA [34] leverages function secret sharing with GPU acceleration to reduce latency for GELU, SiLU, Softmax, and LayerNorm. SHAFT [35] introduces constant-round Softmax and high-precision GELU with reduced communication. East [36] accelerates GELU through oblivious piecewise polynomial evaluation and optimizes Softmax and LayerNorm using conversion and Newton's method. Recently, Mosformer [37] presents the first maliciously secure 3 PC inference framework for large transformer models such as BERT and GPT. Other works, including SecBERT [38], Privformer [22], SecFormer [39], further explore MPC-based privacy-preserving inference.

C. Hybrid HE and MPC Scheme

Since HE and MPC each have their respective strengths in handling linear and non-linear operations, several studies have explored hybrid protocols combining the two. IRON [21] introduces a customized HE-based matrix multiplication protocol with compact packing and optimized protocols for Softmax, GELU, and LayerNorm. Primer [40] uses HE for polynomial layers and MPC for others, shifting most HE computation offline to reduce latency. BOLT [24] improves HE-based matrix multiplication using compact packing and baby-step giant-step strategies, while providing efficient polynomial approximations for GELU, Tanh, and Softmax. BumbleBee [19] introduces an optimized HE-based matrix multiplication protocol and activation protocols. More recently, BLB [41] presents the first secure CKKS–MPC conversion protocol, enabling CKKS-based fused operator computation and a rotation-efficient matrix multiplication algorithm.

III. Preliminaries

A. Transformer Architecture

The transformer architecture consists of an input embedding layer followed by a stack of blocks, each comprising an attention mechanism, a feed-forward network (FFN), and LayerNorm components [20], as shown in Fig. 1. From a protocol design perspective, transformer computations can be classified into linear operations (matrix multiplications) and non-linear operations.

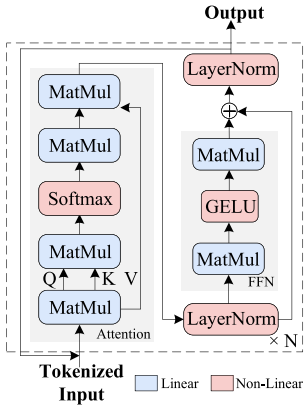


Fig. 1. Transformer architecture.

Matrix Multiplication: Matrix multiplication is used to compute the query Q, key K, value V:

$$Q = XW_Q, K = XW_K, V = XW_V \tag{1}$$

where $X \in \mathbb{R}^{m\times n}$ is the input matrix, $W_Q, W_K, W_V, W_O \in \mathbb{R}^{n\times k}$ are the corresponding weight matrices. Once Q and K are obtained, the attention scores are computed as:

$$Score = QK^T, Attn = SV \tag{2}$$

where $S = softmax(\frac{QK^T}{\sqrt{d_k}})$. The multi-head output is computed as:

$$MultiAttn = Attn \cdot W_O \tag{3}$$

Also, the FFN applies two successive linear transformations with a non-linear activation in between:

$$FFN(X) = GELU(XW_1 + b_1)W_2 + b_2 \tag{4}$$

with W_1, W_2 as weight matrices and b_1, b_2 as bias vectors.

Non-linear Function Evaluation: Non-linear operations in transformers primarily include the following components:

GELU in the FFN:

$$GELU(x) = 0.5x \cdot \left(1 + erf\left(\frac{x}{\sqrt{2}}\right)\right) \tag{5}$$

where $erf(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$.

Softmax in the attention mechanism:

$$Softmax(x)[i] = \frac{\exp(x[i] - \bar{x})}{\sum_j \exp(x[j] - \bar{x})} \tag{6}$$

where $\bar{x}$ is the maximum of vector x.

LayerNorm is applied between the attention mechanism and the FFN to normalize activations across feature dimensions:

$$LayerNorm = \gamma \cdot \frac{(x_i - \mu)}{\sqrt{\sigma^2 + \varepsilon}} + \beta \tag{7}$$

where μ and σ are the mean and standard deviation, γ, β are learnable parameters, and ε is a small constant.

Furthermore, the LLaMA model incorporates two additional non-linear functions:

$$SiLU = x \cdot \sigma(x) \tag{8}$$

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on November 08,2026 at 04:12:15 UTC from IEEE Xplore. Restrictions apply.

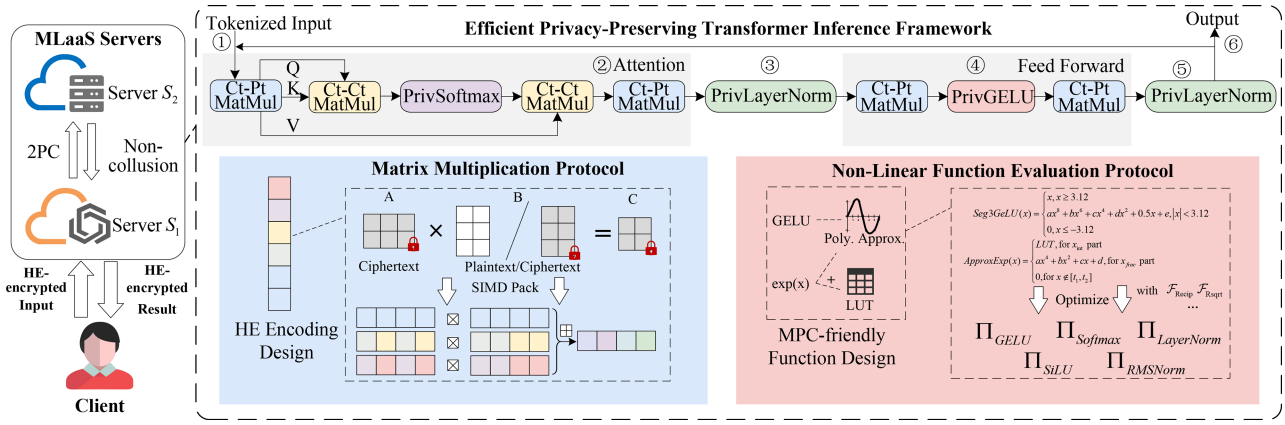


Fig. 2. Overview of PrivTI. The framework consists of our proposed HE-based matrix multiplication protocol and MPC-based non-linear function evaluation protocol to enable efficient privacy-preserving inference for transformer-based models.

where $\sigma(x) = \frac{1}{1+\exp(-x)}$. And

$$RMSNorm = \gamma \cdot \frac{x}{RMS(x)} \tag{9}$$

where $RMS(x) = \sqrt{\frac{\sum_{i=1}^{n} x_i^2}{n} + \varepsilon}$.

B. Cryptographic Primitives

Homomorphic Encryption: HE enables computation directly on encrypted data, producing results equivalent to plaintext operations after decryption.

To enhance computational efficiency, HE supports SIMD-style operations, which encode multiple data elements into a polynomial and leverages the number theoretic transform (NTT) to store them in distinct coefficient positions (slots). Each slot serves as an independent computation domain, enabling parallel operations across all slots. The Brakerski-Fan-Vercauteren (BFV) scheme [42] is an RLWE-based integer HE scheme that supports the following key operations:

1) *Encryption/Decryption:* Given a public key pk and a secret key sk, a plaintext m is encrypted as ciphertext $\tilde{c} = \mathrm{Enc}(m)$ and decrypted as $m = \mathrm{Dec}(\tilde{c})$.
2) *SIMD Encoding:* A plaintext or ciphertext vector $a = [a_0, \ldots, a_{N-1}]$ is packed into a single ciphertext, enabling parallel homomorphic operations across all slots. In BFV, the number of SIMD slots is equal to the polynomial modulus degree N.
3) *Homomorphic Addition ($\boxplus$) / Multiplication ($\boxtimes$):* Ciphertexts support element-wise addition $\tilde{c}_3 = \tilde{c}_1 \boxplus \tilde{c}_2$ and multiplication $\tilde{c}_3 = \tilde{c}_1 \boxtimes \tilde{c}_2$, corresponding to the same operations over plaintext slots.
4) *Rotation (Rot):* A slot-wise cyclic shift over SIMD slots, denoted by $\tilde{c}^{\langle k \rangle} = \mathrm{Rot}(\tilde{c}, k)$.

Secure Multi-Party Computation: MPC allows n parties to jointly compute a public function $y = f(x_1, \ldots, x_n)$ while keeping each input x_i private [9]. PrivTI adopts 2-out-of-2 additive secret sharing (SS) and oblivious transfer (OT) for non-linear function evaluation.

1) *Secret Sharing:* In 2-out-of-2 additive secret sharing, the secret $x \in \mathbb{Z}_{2^l}$ is split into two additive shares held by parties P_1 and P_2, such that $[[x]]_i (i \in \{1, 2\})$ where $[[x]] = [[x]]_1 + [[x]]_2 \pmod{2^l}$, where l denotes the bit length.
2) *Oblivious Transfer:* The sender holds a message vector $(m_0, \ldots, m_{k-1})$, and the receiver chooses an index $b \in \{0, \ldots, k-1\}$ to obtain m_b. The receiver learns nothing about other messages while the sender learns nothing about b.

IV. Efficient Privacy-Preserving Inference Framework for Transformer-Based Models

A. Overview of PrivTI

We propose PrivTI, an efficient end-to-end privacy-preserving inference framework for transformer-based models in MLaaS. It leverages HE for matrix multiplications and MPC for non-linear function evaluation, enabling efficient communication and low-latency privacy-preserving inference. As illustrated in Fig. 2, PrivTI involves two parties:

- *Client:* client C holds the private input for model inference. It locally encrypts the input using HE and transmits the ciphertexts to the MLaaS servers for privacy-preserving inference.
- *MLaaS Servers:* the MLaaS servers host the transformer-based model and provide inference services with sufficient computational and storage resources. Internally, two non-colluding servers, S_1 and S_2, are deployed: S_1 hosts the model parameters w and executes HE-based linear protocols, while S_2 collaborates with S_1 to execute 2PC-based non-linear protocols. This two-server architecture significantly reduces computational overhead compared to pure HE-based approaches and prevents leakage of the client's input.

Considering the distinct computational characteristics of matrix multiplication and non-linear function evaluation in transformer-based models, PrivTI adopts a hybrid HE-MPC scheme. The BFV scheme is employed in HE for efficient matrix multiplication, and 2-out-of-2 additive secret sharing is used in MPC for non-linear function evaluation, with conversions

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on November 08,2026 at 04:12:15 UTC from IEEE Xplore. Restrictions apply.

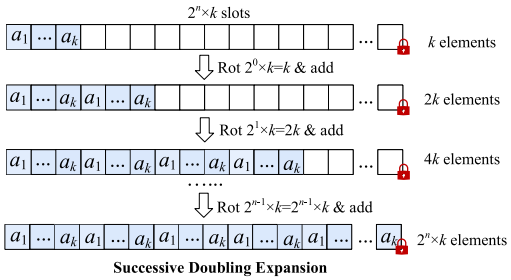


Fig. 3. Successive doubling expansion for matrix A when computing matrix multiplication $A \times B$, where $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times k}$.

between HE and MPC performed as needed. The workflow of PrivTI is described in Section VII-B.

B. Threat Model

We consider the semi-honest setting for privacy-preserving inference. In our architecture, client C holds sensitive input data x and is responsible only for HE encryption and decryption. Server S_1 hosts the model parameters w and executes the HE-based linear protocols, while server S_2 acts as the MPC computation node and jointly performs 2PC-based non-linear protocols with S_1 for privacy-preserving inference.

Under the semi-honest setting, all parties are assumed to follow the protocol honestly but may try to infer additional information from their respective local views. Given the non-collusion assumption between S_1 and S_2, neither server alone can recover the plaintext input x or the inference outcome beyond what is revealed to C via the final output. Since all computations are performed on the MLaaS servers, the model parameters w remain protected from C.

Formally, we assume a static, semi-honest probabilistic polynomial-time (PPT) adversary under the ideal/real world paradigm [43]. In the ideal world, all parties submit their inputs to an ideal functionality $\mathcal{F}$, which faithfully executes the computation and returns outputs to respective parties. In the real world, the protocol Π is executed by the client C and the MLaaS servers (S_1 and S_2) as specified, possibly in the presence of an adversary $\mathcal{A}$, where at most one out of three parties may be corrupted. Security requires that for any adversary $\mathcal{A}$, there exists a simulator Sim such that the adversary's view in the real world execution of Π is computationally indistinguishable from its view in the ideal world execution of $\mathcal{F}$.

C. Design Objective

PrivTI is designed for efficient private transformer inference in MLaaS with minimized communication cost and reduced latency. Its key objectives are:

- *Privacy Preservation:* All computations are performed either in the encrypted domain or via secret sharing, ensuring that the client's sensitive inputs remain confidential throughout the inference process.
- *Communication Efficiency:* Matrix multiplications and non-linear evaluations are structured to enable compact matrix packing, hence substantially reducing communication overhead.
- *Low Latency:* Matrix multiplications and non-linear layers are optimized for efficient protocols, enabling faster computations and reducing inference latency.
- *Comparable Accuracy:* MPC-friendly functions introduce only minor approximation errors compared to the original functions, ensuring inference accuracy that remains comparable to plaintext execution.

V. Matrix Multiplication Protocol With Tailored SIMD Encoding

When HE is applied, computations must be adapted to the characteristics of encrypted operations. Based on the data types involved, the multiplications are categorized into two types: ciphertext–plaintext (Ct-Pt) and ciphertext–ciphertext (Ct-Ct) matrix multiplication. These protocols are executed by S_1.

A. Ciphertext-Plaintext Matrix Multiplication

We first introduce a successive doubling expansion method for elements within SIMD ciphertext slots, which primarily leverages rotation and addition operations on homomorphic ciphertexts to rapidly replicate elements and fill the entire slot.

Successive Doubling Expansion: This method is applied to both Ct-Pt and Ct-Ct matrix multiplication for efficient matrix processing. Starting from k elements, each step performs a rotation by $2^{n-1} \times k$ positions followed by an addition, thereby doubling the number of replicated elements at each iteration. After n iterations, the slots are fully populated with $2^n \times k$ elements. Fig. 3 illustrates the process.

Ct-Pt Matrix Multiplication: These matrix multiplications primarily occur in the computation of $Q/K/V$ and output projections in the attention mechanism, as well as in FFN. Taking the computation of Q, K, and V as an example, let $X \in \mathbb{R}^{m \times n}$ and $W \in \mathbb{R}^{n \times k}$. The server S_1 first packs X into SIMD slots in a column-wise manner, then applies rotation and masking operations to extract each column vector of X, placing it at the beginning of the SIMD slots, resulting in n column-vector ciphertexts. Then, using successive doubling expansion, each column-vector ciphertext is replicated to fill the SIMD slots, producing n processed ciphertexts $\tilde{x}_i$ ($i \in [1, n]$):

$$\tilde{x}_i = Enc\left(\left[\overbrace{x_{1i}, x_{2i}, \ldots, x_{mi}}^{m}, x_{1i}, x_{2i}, \ldots, x_{mi}, \ldots, x_{1i}, x_{2i}, \ldots, x_{mi}}^{m \cdot k}\right]\right), \quad i \in [1, n] \tag{10}$$

For the weight matrix W, preprocessing can be applied. Specifically, each element in W is replicated k times in plaintext to form a k-fold expanded matrix, which is then row-packed into SIMD slots using batch encoding $E()$, resulting in n SIMD plaintext encodings w_i ($i \in [1, n]$):

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on November 08,2026 at 04:12:15 UTC from IEEE Xplore. Restrictions apply.

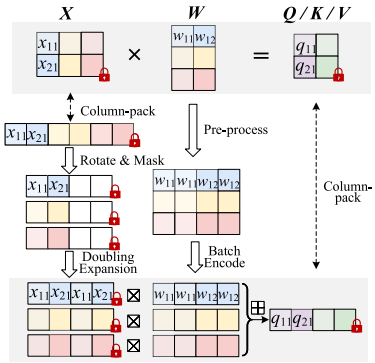


Fig. 4. A toy example of SIMD encoding in Ct-Pt matrix multiplication, where $X \in \mathbb{R}^{2\times 3}$ and the weight matrix $W \in \mathbb{R}^{3\times 2}$.

$$w_i = E\left(\left[\overbrace{w_{i1}, w_{i1}, \ldots, w_{i1}}^{m}, w_{i2}, w_{i2}, \ldots w_{i2}, \ldots, w_{ik}, w_{ik}, \ldots, w_{ik}\right]_{mk}\right), \quad i \in [1, n] \tag{11}$$

Note that the slots of column-packed SIMD ciphertexts $\tilde{x}_i$ and row-packed SIMD plaintext encodings w_i are aligned in a one-to-one manner. By multiplying the ith ($i \in [1, n]$) ciphertext $\tilde{x}_i$ with w_i and summing over all n products, a single SIMD ciphertext is obtained, which represents the resulting matrix in column-major packed form. Fig. 4 shows a toy example of Ct-Pt matrix multiplication.

B. Ciphertext-Ciphertext Matrix Multiplication

Ct-Ct matrix multiplications mainly occur in the attention mechanism, specifically in the computations $Q \times K^T$ and $S \times V$. Since Q, K, and V are computed using the same procedure (Section V-A), they are encoded as column-packed SIMD ciphertexts. Consequently, $Q \times K^T$ involves a column-packed matrix multiplied by a row-packed matrix, while $S \times V$ involves a column-packed matrix multiplied by another column-packed matrix. Therefore, we design separate SIMD encoding methods for $Q \times K^T$ and $S \times V$.

Element-wise Scattering: We first introduce an element-wise scattering method, which is used in Ct-Ct matrix multiplication along with successive doubling expansion. This method uses multiple rotations, additions, and a single masking operation to quickly scatter elements into designated slot positions. Starting with k elements in the SIMD ciphertext slots, the method performs $k \times (2^n - 1)$ iterations of rotations and additions, followed by a masking operation to zero out undesired positions, leaving the k elements scattered into their designated slots. Fig. 5 illustrates the process.

$Q \times K^T$: Since Q and the input matrix X share the same column-packing format, the encoding method for X described in Section V-A can be directly applied here to generate n SIMD-extended ciphertexts $\tilde{q}_i$ ($i \in [1, n]$). For matrix K^T, a different encoding method is used. Assuming $Q \in \mathbb{R}^{m\times n}$ and

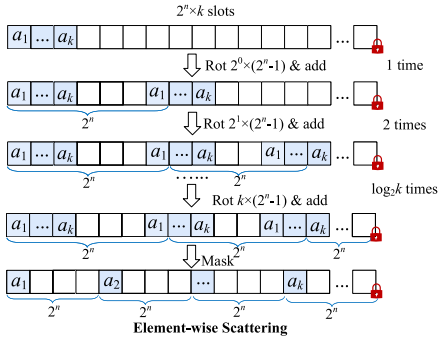


Fig. 5. Element-wise scattering for matrix A when computing the matrix multiplication $A \times B$, where $A \in \mathbb{R}^{m\times n}$ and $B \in \mathbb{R}^{n\times k}$.

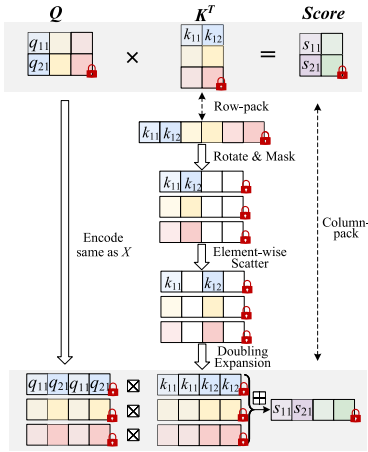


Fig. 6. A toy example of SIMD encoding in $Q \times K^T$, where $Q \in \mathbb{R}^{2\times 3}$ and $K^T \in \mathbb{R}^{3\times 2}$.

$K^T \in \mathbb{R}^{n\times k}$, we first separate the n row vectors of K^T through rotation and masking operations, then apply element-wise scattering to distribute their elements to specific positions in the SIMD slots. Next, successive doubling expansion is used to replicate and fill the slots, producing n ciphertexts $\tilde{k}_i$ ($i \in [1, n]$). Finally, the ith ciphertexts $\tilde{q}_i$ and $\tilde{k}_i$ are multiplied pairwise, and the results are summed up. A toy example of $Q \times K^T$ is given in Fig. 6.

$S \times V$: The matrix S adopts the same encoding method as for input matrix X in Section V-A to obtain n expanded ciphertexts $\tilde{s}_i$ ($i \in [1, n]$). The matrix V uses a different encoding method. Assuming $S \in \mathbb{R}^{m\times n}$ and $V \in \mathbb{R}^{n\times k}$, we first perform masking operations on the ciphertexts of matrix V to separate the row elements in SIMD ciphertext slots. Note that it is unnecessary to rotate all ciphertexts to the head of the SIMD slots, because after masking, these elements are already in their designated positions. Therefore, we directly duplicate the separated ciphertexts by multiplication, which produces n expanded ciphertexts $\tilde{v}_i$ ($i \in [1, n]$). Finally, the ith corresponding ciphertexts $\tilde{s}_i$ and $\tilde{v}_i$ are multiplied pairwise, and the results are summed up, as shown in Fig. 7.

Table I shows the complexity comparison of linear computations across prior works, including the number of multiplications, rotations, and the amount of ciphertext transmitted. It is evident that our PrivTI is more efficient in terms of multiplication

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on November 08,2026 at 04:12:15 UTC from IEEE Xplore. Restrictions apply.

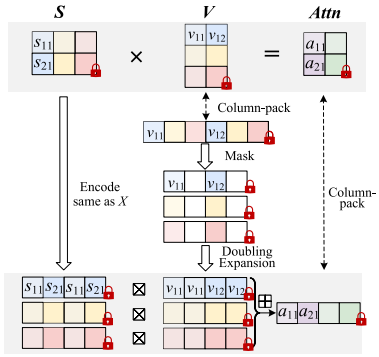


Fig. 7. A toy example of SIMD encoding in $S \times V$, where $S \in \mathbb{R}^{2\times 3}$ and $V \in \mathbb{R}^{3\times 2}$.

TABLE I
THEORETICAL COMPLEXITY COMPARISON OF LINEAR COMPUTATION ACROSS PRIOR WORKS. "/" DENOTES *NOT SUPPORTED* OR *NOT MENTIONED* IN THE PAPER. THE COMPARISON IS BASED ON DIMENSIONS $(m, N, K) = (128, 768, 64)$, AND POLYNOMIAL DEGREE $N = 8192$.

Method	$X \times W$		$Q \times K^\top$		$S \times V$		Transmission
	Mul	Rot	Mul	Rot	Mul	Rot	
GAZELLE [11]	$O(mn)$	$O(mn)$	/	/	/	/	$O(\frac{mn}{k})$
	98304	96768	/	/	/	/	1664
IRON [21]	$O(\frac{mnk}{N})$	0	/	/	/	/	$O(\sqrt{\frac{mnk}{N}})$
	768	0	/	/	/	/	56
BOLT [24]	$O(n)$	$O(\sqrt{\frac{m^2n^2k}{N^2}})$	$O(\frac{m^2n}{N})$	$O(\frac{mn(m+\log_2 m)}{N})$	$O(\frac{m^2n}{N})$	$O(\frac{m^2(m+\log_2 m)}{N})$	$O(\frac{m(n+k)}{N})$
	768	96	3072	1728	1536	3570	13
NEXUS [25]	$O(mn)$	$O(\frac{m^2n}{N})$	$O(\frac{m^2k}{N})$	$O(\frac{m^2k}{N})$	/	/	$O(\frac{mn}{N})$
	98304	6144	512	512	/	/	12
Ours	$O(n)$	$O(n\log_2(\frac{N}{m}))$	$O(k)$	$O(k\log_2 N + (m-k)\log_2 m)$	$O(m)$	$O(m\log_2(\frac{n^2}{mk}))$	$O(\frac{mn}{N})$
	768	5376	64	1792	128	1792	12

count for $X \times W, Q \times K^T$ and $S \times V$. Although our approach involves more rotations than BOLT due to ciphertext processing, it still achieves better overall performance on $Q \times K^T$ and $S \times V$ thanks to the successive doubling expansion and element-wise scattering strategies.

VI. NON-LINEAR FUNCTION EVALUATION PROTOCOL WITH OPTIMIZED MPC-FRIENDLY FUNCTIONS

We choose MPC for non-linear function evaluation, leveraging polynomial approximation and lookup tables to construct MPC-friendly functions. These protocols are jointly executed by S_1 and S_2.

A. Private GELU Protocol

GELU function is relatively complex, and a single function approximation cannot achieve sufficient accuracy. To address this, a piecewise function approximation is adopted to improve fitting accuracy.

GELU approximation: First, when input x is greater than a positive threshold, the value of GELU approaches x, and when x is less than a negative threshold, the value of GELU approaches 0. For the middle region, we note that:

$$GELU(x) = 0.5x + P(x) \tag{12}$$

where $P(x)$ is an even function. We observe that the number of polynomial segments directly impacts interaction rounds and runtime in GELU evaluation, and high-order polynomials can reduce segment count. In addition, the cost of a squaring operation in MPC is half that of a standard multiplication [19], we approximate the $P(x)$ term in GELU using a high-order even polynomial.

The piecewise function adopted in this work requires only three segments to approximate GELU: when $x > 3.12$, $GELU(x) \approx x$. When $x < -3.12$, $GELU(x) \approx 0$. For $x \in (-3.12, 3.12)$, we approximate the $P(x)$ term in GELU with a high-order even polynomial of the form:

$$G(x) = 0.5x + P(x) \approx ax^8 + bx^6 - cx^4 + dx^2 + 0.5x + e \tag{13}$$

This piecewise design simplifies non-linear evaluation under cryptographic protocols. With only three segments, segmentation requires two comparisons. The even high-order terms enable efficient computation via repeated squaring. For example, x^8 is inexpensive and the x^6 term requires a single multiplication $x^2 \times x^4$. The full MPC-friendly $Seg3GELU(x)$ is presented in (14), shown at the bottom of the next page.

Private GELU protocol: We design an MPC-friendly GELU protocol with only two comparisons and few multiplications, reducing communication and latency. Specifically, powers x^2, x^4, and x^8 are computed via repeated squaring, and x^6 is computed by another multiplication. Next, two secure less-than comparisons determine the input interval, after which the polynomial $G(x)$ is securely evaluated for the middle interval, and secure multiplexers combine the three candidate outputs to produce the final approximation.

The complete private GELU protocol is presented in Algorithm 1. We use Π_{mul} for multiplication, Π_{square} for squaring, Π_{LT} for less-than operations, Π_{MulBA} for multiplication between Boolean and arithmetic shares. $[[b]]^B$ denotes Boolean secret sharing. The entire computation is carried out on secret shares: the server S_1 holds secret $[[x]]_C$, and the server S_2 holds secret $[[x]]_S$, where $[[x]] = [[x]]_C + [[x]]_S \pmod{2^l}$ with l representing the bit length. All operations are executed under MPC protocols, ensuring that neither party learns the other's share during computation. For convenience, we use $[[x]]$ to denote the pair of secrets $([[x]]_C, [[x]]_S)$.

B. Private Softmax Protocol

When performing the Softmax operation, the computation of $\exp(x)$ and reciprocal poses significant challenges under cryptographic protocols. To address this, we analyze the computational characteristics of $\exp(x)$ in detail and design optimizations specifically tailored for its use in Softmax.

Exponential function: In Softmax, we observe that inputs during transformer inference typically fall within $(-\infty, 0]$ and $\exp(x)$ varies rapidly across integers but smoothly within (0,1). We therefore decompose the input x into its integer part x_{int} and fractional part x_{frac}, allowing the computation to be split into two components: a small lookup table (LUT) for x_{int} and a lightweight polynomial approximation for the fractional part x_{frac}. Thus, the exponential function can be factorized as $\exp(x) = \exp(x_{int}) \cdot \exp(x_{frac})$.

For the LUT, we store the exponential values for integer inputs $x \in [n, 0]$. Since $\exp(x)$ decays rapidly to near zero for

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on November 08,2026 at 04:12:15 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Private GELU Protocol.

Input: secret shared input $[\![x]\!]$, approximation polynomial $Seg3GELU(x)$
Output: secret shared value $[\![y]\!]$
Function *PrivGELU()*:
  S_1 and S_2 hold their shares and jointly compute $[\![x^2]\!] = \Pi_{square}([\![x]\!])$, $[\![x^4]\!] = \Pi_{square}([\![x^2]\!])$, $[\![x^6]\!] = \Pi_{mul}([\![x^2]\!], [\![x^4]\!])$, $[\![x^8]\!] = \Pi_{square}([\![x^4]\!])$ // under the MPC protocol
  jointly compute $[\![b_0]\!]^B = \Pi_{LT}(-3.12, [\![x]\!])$, $[\![b_1]\!]^B = \Pi_{LT}(3.12, [\![x]\!])$
  // $b_0 = 1\{x > -3.12\}, b_1 = 1\{x > 3.12\}$
  and jointly compute $[\![z_0]\!]^B = [\![b_0]\!]^B \oplus [\![b_1]\!]^B$, $[\![z_1]\!]^B = [\![b_1]\!]^B$
  // $z_0 = 1\{-3.12 < x < 3.12\}$, $z_1 = 1\{x > 3.12\}$
  jointly compute $[\![G(x)]\!]$ based on $[\![x]\!]$ $[\![x^2]\!]$ $[\![x^4]\!]$ $[\![x^6]\!]$ $[\![x^8]\!]$
  jointly compute $[\![y]\!] = \Pi_{MulBA}([\![z_0]\!]^B, [\![G(x)]\!]) + \Pi_{MulBA}([\![z_1]\!]^B, [\![x]\!]))$ and get secret share $[\![y]\!]$
  return $[\![y]\!]$
End

sufficiently small x, we truncate the table at $n = -10$, with values beyond this point approximated as 0. This significantly reduces the table size, thereby minimizing both communication cost and memory usage.

Additionally, we adopt an OT-based LUT protocol to ensure oblivious table access without leaking the private index. Given a private input x and a lookup table $T = \{t[i]\}_{i=0}^{L-1}$, the protocol first computes equality indicators $b_i = 1(x = i)$ for all $i \in \{0, \ldots, L-1\}$ using OT-based secure comparison. Each indicator is then multiplied by the corresponding table entry via OT-based secure multiplication to obtain $y_i = b_i \cdot t[i]$. The final output is computed as $T(x) = \sum_{i=0}^{L-1} y_i$. Since every table entry is processed in the same manner, the access pattern is independent of x, thereby ensuring obliviousness.

For the polynomial approximation, we adopt an even-order polynomial, which enables efficient evaluation under MPC via fast squaring. The resulting polynomial approximation $FracExp(x)$ is given as follows:

$$FracExp(x) = 0.1384768486x^4 + 0.5993350639x^2 + 0.9802810011x + 1.0006317416 \quad (15)$$

Through the above factorization of $\exp(x)$ into integer and fractional components, the evaluation process is significantly simplified and accelerated. The full evaluation protocol is presented in Algorithm 2.

Algorithm 2: Exponential Function Evaluation.

Input: input value $x < 0$, threshold n, lookup table T, approximation polynomial $FracExp(x)$
Output: value $ApproxExp$
Function *ExpEval()*:
  if $x < n$ **then**
    // check the threshold
    return 0
  end
  compute $x_{\text{int}} = \lfloor x \rfloor$, $x_{frac} = x - x_{\text{int}}$ // decompose integer and fraction
  retrieve $E_{\text{int}} = T(x_{\text{int}})$ from the lookup table
  compute $E_{frac} = FracExp(x_{frac})$ using the polynomial approximation
  compute $ApproxExp = E_{\text{int}} \cdot E_{frac}$ // combine results
  return $ApproxExp$
End

Private Softmax protocol: In addition to the optimized exponential evaluation, we adopt a reciprocal protocol consistent with SIRNN [6] to replace the division operation in Softmax with a combination of reciprocal and multiplication. Building on these designs, we formulate a complete private Softmax protocol for efficient privacy-preserving inference, as shown in Algorithm 3. We use Π_{Recip} to denote the reciprocal operation.

C. Private LayerNorm Protocol

We follow the approach in NEXUS [25] to simplify the computation of LayerNorm. Let $y_i = nx_i - n\mu = nx_i - \sum_{i=1}^{n} x_i$, and we compute LayerNorm as:

$$LayerNorm = \sqrt{n} \cdot \gamma \cdot \frac{y_i}{\sqrt{\sum_{i=1}^{n} {y_i}^2 + \varepsilon}} + \beta \quad (16)$$

To optimize the reciprocal square root operation, we adopt Newton's iteration method. Let $f(x) = \frac{1}{x^2} - a = 0$, so $x = \frac{1}{\sqrt{a}}$. According to Newton's Iteration formula $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$, we derive the iterative update function: $x_{n+1} = \frac{x_n}{2}(3 - ax_n^2)$. We choose $2^{-\lfloor \alpha/2 \rfloor}$ as an initial guess with $2^{\alpha} \le x \le 2^{\alpha+1}$. The reciprocal square root is denoted by Π_{Rsqrt}. Algorithm 4 shows our private LayerNorm protocol.

$$Seg3GELU(x) = \begin{cases} x, x \ge 3.12 \\ G(x) = -0.0001506671x^8 + 0.0043702779x^6 - 0.0512144957x^4 + 0.3828738634x^2 + 0.5x + 0.0028445838, |x| < 3.12 \\ 0, x \le -3.12 \end{cases} \quad (14)$$

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on November 08,2026 at 04:12:15 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3: Private Softmax protocol.

Input: secret shared vector $[\![x]\!] = ([\![x_1]\!], [\![x_2]\!], ..., [\![x_n]\!])$
Output: secret shared vector $[\![s]\!] = ([\![s_1]\!], [\![s_2]\!], ..., [\![s_n]\!])$

1 **Function** *PrivSoftmax()*:
2 S_1 and S_2 hold their shares and jointly compute $\bar{x} = \max([\![x_1]\!], [\![x_2]\!], ..., [\![x_n]\!])$ // under the MPC protocol
3 **for** $i = 1$ *to* n **do**
4 $[\![z_i]\!] = [\![x_i]\!] - \bar{x}$
5 $[\![ApproxExp_i]\!] = ExpEval([\![z_i]\!])$
6 **end**
7 jointly compute $[\![sum]\!] = \sum_{i=1}^{n} [\![ApproxExp_i]\!]$
8 jointly compute $[\![\frac{1}{sum}]\!] = \Pi_{Recip}([\![sum]\!])$
9 **for** $i = 1$ *to* n **do**
10 $[\![s_i]\!] = [\![ApproxExp_i]\!] \cdot [\![\frac{1}{sum}]\!]$
11 **end**
12 **return** $[\![s]\!] = ([\![s_1]\!], [\![s_2]\!], ..., [\![s_n]\!])$
13 **End**

Algorithm 4: Private LayerNorm protocol.

Input: secret shared vector $[\![x]\!] = ([\![x_1]\!], [\![x_2]\!], ..., [\![x_n]\!])$, parameters β and γ, a small constant ε
Output: secret shared vector $[\![\ln]\!] = ([\![\ln_1]\!], [\![\ln_2]\!], ..., [\![\ln_n]\!])$

1 **Function** *PrivLayerNorm()*:
2 S_1 and S_2 hold their shares and jointly compute $[\![sum]\!] = \sum_{i=1}^{n} [\![x_i]\!]$ // under the MPC protocol
3 **for** $i = 1$ *to* n **do**
4 $[\![y_i]\!] = n\,[\![x_i]\!] - [\![sum]\!]$
5 $[\![z_i]\!] = \Pi_{square}[\![y_i]\!]$
6 $[\![a]\!] = \sum_{i=1}^{n} [\![z_i]\!] + \varepsilon$
7 $[\![t]\!] = \Pi_{Rsqrt}([\![a]\!])$
8 $[\![\ln_i]\!] = \sqrt{n} \cdot \gamma \cdot [\![y_i]\!] \cdot [\![t]\!] + \beta$
9 **end**
10 **return** $[\![\ln]\!] = ([\![\ln_1]\!], [\![\ln_2]\!], ..., [\![\ln_n]\!])$
11 **End**

D. Extension to Private SiLU and RMSNorm Protocols

Considering that widely used transformer models, such as LLaMA, adopt SiLU instead of GELU and RMSNorm instead of LayerNorm, we further extend our protocols to support these operations.

The main challenge in evaluating SiLU lies in the computation of exponential and reciprocal functions. However, a direct polynomial approximation of SiLU fails to achieve high accuracy without increasing either the number of segments or the polynomial degree. To address this, we refine the Softmax computation strategy introduced in Section VI-B, enlarging the LUT for the integer part to cover $x \in [-10, 8]$ for efficient and scalable evaluation of SiLU.

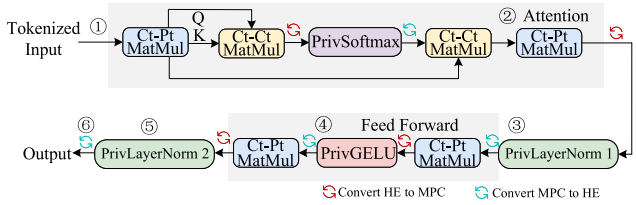


Fig. 8. End-to-end privacy-preserving inference.

In Section VI-C, we observe that RMSNorm closely follows the computational flow of LayerNorm. Therefore, we reuse Algorithm 4 by mapping the input x to y, modifying Line 6 to normalize the sum by the number of elements n, and revising Line 8 according to the RMSNorm definition. With these minor modifications, RMSNorm can be securely evaluated.

VII. End-to-End Privacy-Preserving Inference

A. Conversion Between HE and MPC

PrivTI employs hybrid HE–MPC protocols to enable privacy-preserving inference. As a result, ciphertexts must be converted between HE over the prime field $\mathbb{Z}_p$ and MPC over the ring $\mathbb{Z}_{2^l}$, where p denotes a large prime number and l denotes the bit length. PrivTI adopts a conversion approach similar to that used in FASTLMPI [44]. When converting from HE to MPC, the process first checks whether the sum exceeds the modulus to compute a binary wrap-bit, then performs a Boolean-to-Arithmetic (B2A) conversion, finally adjusts the original shares. This securely extends the shares from the prime field $\mathbb{Z}_p$ to the ring $\mathbb{Z}_{2^l}$. When converting from MPC to HE, a local-truncation-like probabilistic conversion is applied. The probability of sharing overflow is $\frac{|x|}{2^l}$, where $|x|$ denotes the absolute value of the shared secret. By ensuring $|x|$ is sufficiently small or by choosing a large enough ring size 2^l, the conversion step can be safely omitted without compromising accuracy.

B. Efficient End-to-End Privacy-Preserving Inference

PrivTI performs end-to-end privacy-preserving transformer inference via the sequence of steps given in Fig. 8.

Input phase: The client C encrypts its private input via HE and transmits the ciphertext to the server S_1.

Attention layer: The server S_1 computes query (Q), key (K), and value (V) matrices via Ct-Pt matrix multiplication under HE. Then, it performs a Ct-Ct matrix multiplication between Q and K^T to compute attention scores. Next, it converts the intermediate results from HE domain to MPC domain, and securely computes Softmax under MPC with S_2, followed by the MPC-to-HE conversion. The result is multiplied with V through another Ct-Ct matrix multiplication under HE by S_1 to generate the attention output.

LayerNorm layer 1: The HE ciphertext is converted to MPC domain, where a privacy-preserving LayerNorm protocol is executed under MPC between S_1 and S_2 to securely compute normalized outputs.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on November 08,2026 at 04:12:15 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMPARISONS WITH BOLT USING WORD ELIMINATION AND NEXUS USING GPU ACCELERATION

Operation	Setting	Method	Time (s)					Comm. (MB)
			LAN	WAN_1	WAN_2	WAN_3	WAN_4	
Ct-Pt MatMul 128×768×64	Word Elimination	BOLT [24]	5.52	5.78	5.66	5.70	5.75	3.53
		Ours	**2.47**	**2.59**	**2.65**	**3.02**	**3.12**	**3.01**
	GPU	NEXUS [25]	18.83	18.83	18.83	18.83	18.83	/
		Ours	**1.42**	**1.71**	**1.79**	**2.02**	**2.05**	**6.02**
GELU 128×3072	Word Elimination	BOLT [24]	**2.44**	**35.34**	**47.31**	**63.47**	**69.00**	**367.91**
		Ours	3.49	51.31	64.00	94.41	100.24	551.53
	GPU	NEXUS [25]	**2.1**	**2.1**	**2.1**	**2.1**	**2.1**	/
		Ours	6.68	99.36	111.35	194.42	201.02	1141.50
Softmax 128×128	Word Elimination	BOLT [24]	**0.78**	21.49	41.32	21.24	**40.88**	**15.84**
		Ours	0.85	**20.80**	**40.40**	**20.09**	41.47	18.66
	GPU	NEXUS [25]	**1.2**	**1.2**	**1.2**	**1.2**	**1.2**	/
		Ours	1.21	23.46	45.68	26.37	47.24	78.57
LayerNorm 128×768	Word Elimination	BOLT [24]	1.96	35.38	57.80	54.92	73.72	**217.86**
		Ours	**1.79**	**35.34**	**57.74**	**50.72**	**69.87**	217.87
	GPU	NEXUS [25]	**2.0**	**2.0**	**2.0**	**2.0**	**2.0**	/
		Ours	2.84	51.11	74.61	84.43	105.32	442.17

Feed-forward layer: The result is converted back to HE domain and passed through a fully connected layer via Ct-Pt matrix multiplication by S_1. It is then converted to MPC for GELU activation, which is jointly evaluated by S_1 and S_2 using MPC-friendly approximations. The output is converted back to HE domain and undergoes another Ct–Pt matrix multiplication by S_1 to produce the next-layer input.

LayerNorm layer 2: The ciphertext is again converted to MPC domain, and LayerNorm is securely and jointly executed by S_1 and S_2. The output is either forwarded to the next transformer layer or taken as the final output.

Output phase: The final result is converted back to HE domain by S_1 and is sent back to the client C, who decrypts locally to recover the plaintext inference result.

By jointly designing the matrix multiplication protocol and the non-linear function evaluation protocol, PrivTI achieves efficient communication and accelerated computation in privacy-preserving transformer inference, and produces results nearly identical to plaintext inference.

VIII. EVALUATIONS

A. Experimental Setup

PrivTI is implemented in C++, leveraging the Secure and Correct Inference (SCI) library[1] from EzPC for MPC and the Microsoft SEAL library[2] for HE. We have open-sourced our code on GitHub.[3]

We use the BFV scheme with a polynomial modulus degree $N = 8192$. All experiments are conducted on a machine with Intel Core i7-10700K CPU (3.80GHz, 8 cores, 16 threads), NIVDIA 3080 GPU and 128GB of RAM. We set the number of threads to 16, and each reported result is averaged over five runs. Network bandwidth and latency are controlled using the Linux traffic control (tc) command. We evaluate the performance under five network configurations: LAN (3Gbps bandwidth, 0.8ms latency), WAN_1 (200Mbps bandwidth, 40ms latency), WAN_2 (200Mbps bandwidth, 80ms latency), WAN_3 (100Mbps bandwidth, 40ms latency) and WAN_4 (100 Mbps bandwidth, 80 ms latency).

[1]https://github.com/mpc-msri/EzPC/tree/master/SCI
[2]https://github.com/microsoft/SEAL
[3]https://github.com/isaac1216/PrivTI

TABLE III
PERFORMANCE OF PRIVATE SILU AND RMSNORM PROTOCOLS

Operation	Time (s)					Comm. (MB)
	LAN	WAN_1	WAN_2	WAN_3	WAN_4	
SiLU 8×14336	1.94	49.09	59.39	27.55	39.21	290.51
RMSNorm 8×4096	1.45	30.54	48.88	24.06	42.85	105.13

We select three representative prior works as baselines: IRON [21] and BOLT [24], which also build upon EzPC and BFV, and NEXUS [25], which performs end-to-end inference entirely under HE. Fig. 9 presents the baseline performance comparison results with word elimination (a machine learning optimization) disabled in BOLT [24] and GPU acceleration disabled in NEXUS [25]. We provide a further comparison with BOLT utilizing word elimination and NEXUS utilizing GPU acceleration[4] in Table II.

We conduct two types of evaluation: microbenchmarks, analyzing the efficiency of individual protocols, and end-to-end benchmarks, assessing overall inference performance.

B. Microbenchmarks

In the microbenchmark experiments, we use matrices of size $128 \times 768 \times 64$, $128 \times 64 \times 128$, and $128 \times 128 \times 64$ for Ct-Pt matrix multiplication, $Q \times K^T$, and $S \times V$, respectively. For non-linear function evaluation, the input sizes are set to 128×3072 for GELU, 128×128 for Softmax, 128×768 for LayerNorm, 8×14336 for SiLU, and 8×4096 for RMSNorm.

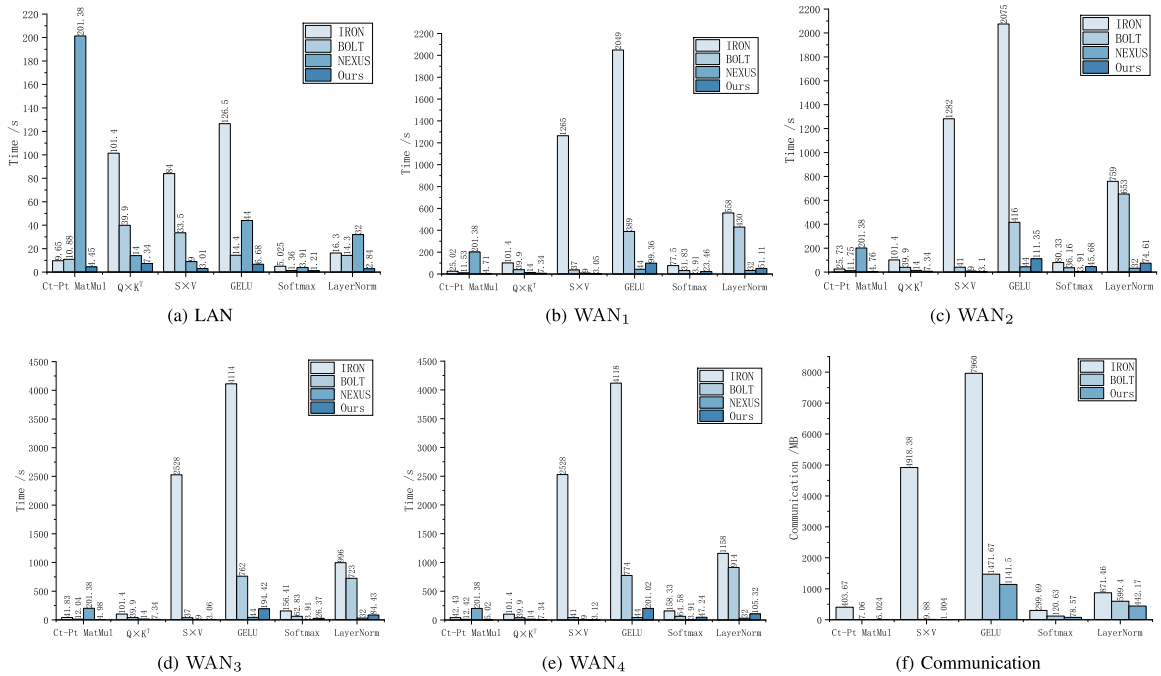
Matrix Multiplication: Our Ct-Pt MatMul method runs in 4.45 s with 6.02 MB communication under the LAN setting, achieving 2.1–45.2× speedup over IRON [21], BOLT [24], and NEXUS [25], while reducing communication by 1.1-67.0× compared to IRON and BOLT. For $Q \times K^T$, the runtime is 3.01s (1.9–13.8× speedup), with no additional communication. Similarly, for $S \times V$, the runtime is 3.01s with 1.00MB communication, yielding 2.9–27.9× speedup and reducing communication by 9.8-4898.7×. Similar advantages are observed under WAN settings, as shown in Fig. 9.

Non-linear Function Evaluation: Our GELU method runs in 6.68 s with 1141.50 MB communication in the LAN setting, achieving 2.1–18.9× speedup over IRON [21], BOLT [24], and NEXUS [25], as well as reducing communication by 1.2-6.9× compared to IRON and BOLT. Our Softmax method runs in 1.21s with 78.57MB communication, yielding 1.1–4.1× speedup, as well as reducing communication by 1.5-3.8×. Our LayerNorm method runs in 2.84s with 442.17MB communication, achieving 5.0–11.2× speedup, as well as reducing communication by 1.3-1.9×. NEXUS incurs the lowest communication due to its non-interactive design and can outperform ours under low-bandwidth or high-latency settings, as shown in Fig. 9. In addition, we evaluate our SiLU and RMSNorm protocols under various network settings, as summarized in Table III.

Others: We compare the maximum error and mean absolute error (MAE) of GELU over $(-5, 5)$ and $\exp(x)$ over (0,1)

[4]https://github.com/encryptorion-lab/phantom-fhe

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on November 08,2026 at 04:12:15 UTC from IEEE Xplore. Restrictions apply.



(a) LAN (b) WAN_1 (c) WAN_2

(d) WAN_3 (e) WAN_4 (f) Communication

Fig. 9. Runtime under different network settings and Communication overhead of linear & non-linear operators.

TABLE IV
ERROR OF POLYNOMIAL APPROXIMATIONS

Operation	Method	Maximum Error	Mean Absolute Error
GELU	BOLT [24]	9.35×10^{-3}	1.18×10^{-3}
	BumbleBee [19]	3.33×10^{-2}	4.06×10^{-3}
	Ours	2.84×10^{-3}	1.15×10^{-3}
$\exp(x)$	BOLT [24]	1.28×10^{-3}	2.13×10^{-3}
	BumbleBee [19]	1.54×10^{-3}	4.25×10^{-3}
	Ours	3.40×10^{-4}	6.32×10^{-4}

TABLE V
PERFORMANCE OF OT-BASED LUT

Operation	Table Length	Time (ms)	Comm. (MB)
LUT (OT-based)	2^4	22.237	5.41
	2^7	176.742	43.27
	2^{10}	1402.02	346.00

against BOLT [24] and BumbleBee [19]. As shown in Table IV, our approximations achieve the lowest errors.

We evaluate the performance of the OT-based LUT with a linear scan over 128×128 elements, as shown in Table V. Since our lookup table corresponds to $y = \exp(x)$ for $x \in \{-10, -9, \ldots, 0\}$, its size is relatively small. Thus, computing $\exp(x)$ via the LUT incurs minimal runtime and communication overheads. We also benchmark the performance of conversions between HE and MPC, as shown in Table VI.

In addition, Table II presents the performance comparison against BOLT [24] utilizing word elimination and NEXUS [25] with GPU acceleration. Since word elimination is a machine learning optimization irrespective of the protocol design, it can also be applied to PrivTI, leading to a reduction of runtime and nearly halving the communication cost. However, since the non-linear operations in PrivTI are MPC-based and not GPU-friendly yet, PrivTI can leverage GPU acceleration only for linear operations. As a result, NEXUS outperforms PrivTI in non-linear operations when GPU acceleration is fully utilized.

TABLE VI
PERFORMANCE OF CONVERSIONS BETWEEN HE AND MPC FOR DIMENSIONS OF 128×3072 (GELU), 128×128 (SOFTMAX), AND 128×768 (LAYERNORM). "PRE-" DENOTES *BEFORE THE OPERATION*, WHILE "POST-" DENOTES *AFTER THE OPERATION*.

Conversion	Position	Time (s)	Comm. (MB)
HE to MPC	Pre-Softmax	0.22	74.29
	Pre-LayerNorm 1	0.07	24.78
	Pre-GELU	0.15	49.67
	Pre-LayerNorm 2	0.14	49.45
MPC to HE	Post-Softmax	0.19	25.18
	Post-LayerNorm 1	0.39	50.35
	Post-GELU	0.40	50.35
	Post-LayerNorm 2	0.20	25.18
Total	12 layers	21.12	4191.00

TABLE VII
ACCURACY OF BERT-BASE ON THE GLUE BENCHMARKS

Model	Dataset	Plaintext	Ours
BERT-base	RTE	70.26%	69.97%
	SST-2	92.11%	91.59%
	QNLI	89.27%	88.64%

TABLE VIII
END-TO-END PERFORMANCE FOR ONE LAYER ON BERT-BASE WITH INPUT SEQUENCE LENGTH OF 128

Method	Time (s)					Comm. (MB)
	LAN	WAN_1	WAN_2	WAN_3	WAN_4	
IRON [21]	3536.6	21107.78	22489.48	41239.84	42218.4	23229.44
BOLT [24]	2202.27	4016.54	4637.11	5530.83	6175.94	6161.92
NEXUS [25]	6558.97	6558.97	6558.97	6558.97	6558.97	13.65
Ours	663.43	1160.01	1499.91	1394.52	1702.44	3722.12

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on November 08,2026 at 04:12:15 UTC from IEEE Xplore. Restrictions apply.

TABLE IX
THE WORKFLOW AND PERFORMANCE BREAKDOWN OF GPT-2-BASE AND LLAMA3-8B

Operations	GPT-2-base (64 sequence)			Llama3-8B (8 sequence)		
	Input	Time(s)	Comm.(MB)	Input	Time(s)	Comm.(MB)
LayerNorm/RMSNorm	$\mathbb{R}^{64\times768}$	2.19	228.44	$\mathbb{R}^{8\times4096}$	1.45	105.13
MatMul	$(\mathbb{R}^{64\times768}\times\mathbb{R}^{768\times768})\times3$	26.71	36.14	$(\mathbb{R}^{8\times4096}\times\mathbb{R}^{4096\times4096})\times3$	85.76	64.25
MatMul	$(\mathbb{R}^{64\times64}\times\mathbb{R}^{64\times64})\times12$	22.02	/	$(\mathbb{R}^{8\times128}\times\mathbb{R}^{128\times8})\times32$	29.76	/
Softmax	$\mathbb{R}^{64\times64}\times12$	12.36	234.84	$\mathbb{R}^{8\times8}\times32$	0.42	12.81
MatMul	$(\mathbb{R}^{64\times64}\times\mathbb{R}^{64\times64})\times12$	9.03	6.02	$(\mathbb{R}^{8\times8}\times\mathbb{R}^{8\times128})\times32$	10.88	16.06
MatMul	$\mathbb{R}^{64\times768}\times\mathbb{R}^{768\times768}$	26.71	36.14	$\mathbb{R}^{8\times4096}\times\mathbb{R}^{4096\times4096}$	85.76	64.25
LayerNorm/RMSNorm	$\mathbb{R}^{64\times768}$	2.19	228.44	$\mathbb{R}^{8\times4096}$	1.45	105.13
MatMul	$\mathbb{R}^{64\times768}\times\mathbb{R}^{768\times3072}$	106.80	3.01	$\mathbb{R}^{8\times4096}\times\mathbb{R}^{4096\times14336}$	300.16	224.89
GELU/SiLU	$\mathbb{R}^{64\times3072}$	3.48	578.32	$\mathbb{R}^{8\times14336}$	2.17	356.30
MatMul	$\mathbb{R}^{64\times3072}\times\mathbb{R}^{3072\times768}$	95.48	12.04	$\mathbb{R}^{8\times14336}\times\mathbb{R}^{14336\times4096}$	297.53	7.02
Total	12 layers	3683.64	16360.68	32 layers	26090.88	30586.88

C. End-to-End Benchmarks

Accuracy: We evaluate the impact of our non-linear function evaluation protocols on inference accuracy with three datasets RTE, SST-2, and QNLI from the GLUE benchmark.[5] As shown in Table VII, our PrivTI achieves accuracy very close to plaintext inference.

End-to-End performance on BERT-base: We evaluate end-to-end performance on BERT-base under different network settings and compare PrivTI with IRON [21], BOLT [24], and NEXUS [25] under the CPU setting. Similar to NEXUS, the overall end-to-end performance is approximately the aggregation of microbenchmark results. Performance for a single BERT-base layer is reported in Table VIII. In the LAN setting, our PrivTI achieves an inference time that is 3.3-9.8× faster than IRON, BOLT, and NEXUS, while reducing communication by 1.6-6.2× compared to IRON and BOLT. NEXUS still achieves the lowest communication overhead owing to its non-interactive design. Overall, these results demonstrate that PrivTI offers superior efficiency compared to these frameworks, showing significant acceleration in computation time and reduction in communication overhead, second only to the non-interactive communication advantage held by NEXUS.

Performance on GPT-2-base and Llama3-8B: We analyze the performance breakdown for larger models GPT-2-base and Llama3-8B, with input sequence lengths set to 64 and 8, respectively. Table IX reports the runtime and communication for each individual operation and the overall results. Results show that each operator contributes to runtime and communication, and that larger hidden size, feed-forward dimension, head count, and layer depth significantly impact privacy-preserving inference performance.

IX. DISCUSSION

Malicious Security: In this work, our threat model assumes a semi-honest client and two non-colluding semi-honest servers. Since the underlying cryptographic building blocks employed in PrivTI are derived from the EzPC framework [9] and have been formally proven secure, the security of PrivTI follows from the security of these validated components under standard composition assumptions. To further achieve malicious security, additional mechanisms would need to be incorporated in PrivTI.

For client-malicious security, zero-knowledge proof (ZKP) can be incorporated during HE ciphertext generation to ensure ciphertext validity, as adopted in MUSE [45] and SIMC [46].

For server-malicious security, assuming that at most one of the two servers is corrupted, additional verification mechanisms are required for linear layers, non-linear layers and HE-MPC conversions. In particular, for HE-based linear computations, ZKP can be introduced at designated critical steps to enforce computational consistency. For example, one may prove that $Y = X \times W$ holds with respect to committed model parameters W, thereby preventing a malicious server from using incorrect model weights. However, since transformer-based models typically involve large matrix dimensions, enforcing consistency for HE-based linear layers may incur substantial computational overhead. For MPC-based non-linear computations, malicious security can be achieved through authenticated secret sharing by attaching message authentication code (MAC) shares and performing MAC checks during reconstruction. This approach is widely adopted in malicious-secure MPC-based inference frameworks such as MP-SPDZ [47] and recent MD-ML [48], MD-SONIC [49] and Mosformer [37]. In addition, HE–MPC conversion requires malicious-secure comparison and multiplexing operations to correctly interpret signed representations, and the converted values should then be re-encoded into authenticated shares to ensure integrity.

However, this work primarily focuses on the semi-honest setting. Extending PrivTI to malicious security would require further considerations, such as determining the appropriate placement of proof generation, specifying the MAC-check protocol, and carefully choosing the timing of consistency checks, along with establishing a formal security proof. Moreover, these extensions would introduce additional computational and communication overhead. Therefore, extending PrivTI to achieve malicious security warrants further investigation.

Scalability: This work focuses on optimizing the underlying operators in transformer-based models, and the approach can theoretically be extended to other models using the same primitives. We evaluate PrivTI under parameter settings derived from BERT-base and GPT-2-base.

For larger transformer-based models with more layers and/or higher hidden dimensions (e.g., LLaMA3-7B), we evaluate only SiLU and RMSNorm with tensor dimensions of 8×14336 and 8×4096, respectively, and provide a breakdown analysis to

[5]https://gluebenchmark.com/

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on November 08,2026 at 04:12:15 UTC from IEEE Xplore. Restrictions apply.

guide optimization for LLaMA-like models. A full implementation requires further optimization and validation.

Privacy-preserving inference incurs significantly higher costs than plaintext inference. Extending such techniques to production-scale models remains a challenging and evolving area of research. Recent studies on privacy-preserving inference for LLaMA models highlight substantial overheads. For example, to generate a single token from an 8-tokens input, BumbleBee [19] reports that evaluating LLaMA-7B requires over 7.96 minutes on 2 cloud servers equipped with 64 vCPUs, while PUMA [23] requires approximately 200 seconds on 3 cloud servers with 128 vCPUs. In the case of SIGMA [34], the function secret sharing (FSS) key size can reach as high as 255.41 GB for LLaMA2-7B.

Therefore, while extending PrivTI to production-scale transformer models is theoretically feasible, the substantial computational and communication overheads pose significant barriers to practical deployment. Mitigating these overheads remains a challenge in privacy-preserving inference, and future advances in algorithm, protocol, and system-level optimization are expected to enhance its practicality.

Bottleneck: Experiments show that linear layers dominate runtime, while non-linear layers dominate communication overhead. This suggests that future optimization should focus on further reducing rotation operations and minimizing communication rounds. Moreover, BLB [41] demonstrates that the CKKS scheme offers clear advantages over BFV in the evaluation of consecutive linear operators, while Mosformer [37] achieves constant-round comparison in the online phase. We plan to explore these directions in our future work.

X. Conclusion

In this work, we present PrivTI, an efficient end-to-end privacy-preserving inference framework for transformer-based models in MLaaS. PrivTI integrates optimized HE-based matrix multiplications and MPC-based non-linear evaluation protocols. Extensive experiments show that PrivTI achieves accuracy comparable to plaintext inference while significantly improving computational and communication efficiency, paving the way for practical privacy-preserving transformer inference.

References

[1] E. Hesamifard, H. Takabi, M. Ghasemi, and R. N. Wright, "Privacy-preserving machine learning as a service," *Proc. Privacy Enhancing Technol.*, vol. 37, no. 6, pp. 214–223, Nov. 2023.

[2] S. Xie, Y. Xue, Y. Zhu, and Z. Wang, "SkyML: A MLaaS federation design for multicloud-based multimedia analytics," *IEEE Trans. Multimedia*, vol. 27, pp. 2463–2476, 2025.

[3] J. Dalzochio et al., "Machine learning and reasoning for predictive maintenance in industry 4.0: Current status and challenges," *Comput. Ind.*, vol. 123, 2020, Art. no. 103298.

[4] IMARC, "Machine learning as a service market size, report 2025–2033," 2024. [Online]. Available: https://www.imarcgroup.com/machine-learning-as-a-service-market

[5] Q. Zhang, T. Xiang, Y. Cai, Z. Zhao, N. Wang, and H. Wu, "Privacy-preserving machine learning as a service: Challenges and opportunities," *IEEE Netw.*, vol. 37, no. 6, pp. 214–223, Nov. 2023.

[6] D. Rathee et al., "SiRnn: A math library for secure RNN inference," in *Proc. IEEE Symp. Secur. Privacy*, 2021, pp. 1003–1020.

[7] D. Rathee et al., "CryptFlow2: Practical 2-party secure inference," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2020, pp. 325–342.

[8] B. Knott, S. Venkataraman, A. Hannun, S. Sengupta, M. Ibrahim, and L. van der Maaten, "CrypTen: Secure multi-party computation meets machine learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 4961–4973.

[9] N. Chandran, D. Gupta, A. Rastogi, R. Sharma, and S. Tripathi, "EZPC: Programmable and efficient secure two-party computation for machine learning," in *Proc. IEEE Eur. Symp. Secur. Privacy*, 2019, pp. 496–511.

[10] Z. Huang, W.-J. Lu, C. Hong, and J. Ding, "Cheetah: Lean and fast secure {Two-Party} deep neural network inference," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 809–826.

[11] C. Juvekar, V. Vaikuntanathan, and A. Chandrakasan, "{GAZELLE}: A low latency framework for secure neural network inference," in *Proc. 27th USENIX Secur. Symp.*, 2018, pp. 1651–1669.

[12] Z. Á. Mann, C. Weinert, D. Chabal, and J. W. Bos, "Towards practical secure neural network inference: The journey so far and the road ahead," *ACM Comput. Surv.*, vol. 56, no. 5, pp. 1–37, 2023.

[13] J. Feng, Y. Wu, H. Sun, S. Zhang, and D. Liu, "Panther: Practical secure 2-party neural network inference," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 1149–1162, 2025.

[14] X. Liu et al., "Antelope: Fast and secure neural network inference," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 6, pp. 7334–7347, Nov./Dec. 2025.

[15] A. Diaa et al., "Fast and private inference of deep neural networks by co-designing activation functions," in *Proc. 33 rd USENIX Secur. Symp.*, 2024, pp. 2191–2208.

[16] W. Ao and V. N. Boddeti, "AutoFHE: Automated adaption of CNNs for efficient evaluation over FHE," in *Proc. 33 rd USENIX Secur. Symp.*, 2024, pp. 2173–2190.

[17] Y. Fu et al., "Swift: Fast secure neural network inference with fully homomorphic encryption," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 2793–2806, 2025.

[18] W. Yang, X. Li, M. Li, Z. Zhang, Z. Guan, and L. Zhu, "VSecNN: Verifiable and privacy-preserving neural network inference in cloud service," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 10541–10555, 2025.

[19] W. Lu et al., "BumbleBee: Secure two-party inference framework for large transformers," in *Proc. 32nd Annu. Netw. Distrib. Syst. Secur. Symp.*, 2025.

[20] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.

[21] M. Hao, H. Li, H. Chen, P. Xing, G. Xu, and T. Zhang, "Iron: Private inference on transformers," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 15718–15731.

[22] Y. Akimoto, K. Fukuchi, Y. Akimoto, and J. Sakuma, "Privformer: Privacy-preserving transformer with MPC," in *Proc. IEEE 8th Eur. Symp. Secur. Privacy*, 2023, pp. 392–410.

[23] Y. Dong et al., "PUMA: Secure inference of LLaMA-7b in five minutes," *Secur. Saf.*, vol. 4, 2025, Art. no. 2025014.

[24] Q. Pang, J. Zhu, H. Möllering, W. Zheng, and T. Schneider, "Bolt: Privacy-preserving, accurate and efficient inference for transformers," in *Proc. IEEE Symp. Secur. Privacy*, 2024, pp. 4753–4771.

[25] J. Zhang et al., "Secure transformer inference made non-interactive," in *Proc. 32nd Annu. Netw. Distrib. Syst. Secur. Symp.*, 2025.

[26] T. Chen et al., "The-X: Privacy-preserving transformer inference with homomorphic encryption," in *Proc. Findings Assoc. Comput. Linguistics: ACL*, 2022, pp. 3510–3520.

[27] L. Rovida and A. Leporati, "Encrypted image classification with low memory footprint using fully homomorphic encryption," *Int. J. Neural Syst.*, vol. 34, no. 5, 2024, Art. no. 2450025.

[28] J. Moon, D. Yoo, X. Jiang, and M. Kim, "THOR: Secure transformer inference with homomorphic encryption," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2025, pp. 3765–3779.

[29] I. Zimerman, M. Baruch, N. Drucker, G. Ezov, O. Soceanu, and L. Wolf, "Converting transformers to polynomial form for secure inference over homomorphic encryption," in *Proc. 41st Int. Conf. Mach. Learn.*, 2024, pp. 62803–62814.

[30] D. Park, E. Lee, and J.-W. Lee, "Powerformer: Efficient and high-accuracy privacy-preserving language model with homomorphic encryption," in *Proc. 63rd Annu. Meeting Assoc. Comput. Linguistics*, 2025, pp. 11090–11111.

[31] Z. He, M. Wang, X. Gao, Y. Luo, L. Liu, and S. Fu, "ENSI: Efficient non-interactive secure inference for large language models," 2025, *arXiv:2509.09424*.

[32] C. Zeng, D. He, Q. Feng, X. Yang, and Q. Luo, "SecureGPT: A framework for multi-party privacy-preserving transformer inference in GPT," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 9480–9493, 2024.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on November 08,2026 at 04:12:15 UTC from IEEE Xplore. Restrictions apply.

[33] Y. Chen, X. Meng, Z. Shi, Z. Ning, and J. Lin, "SecureTLM: Private inference for transformer-based large model with MPC," *Inf. Sci.*, vol. 667, 2024, Art. no. 120429.
[34] K. Gupta et al., "Sigma: Secure GPT inference with function secret sharing," in *Proc. Privacy Enhancing Technol.*, 2024, pp. 61–79.
[35] A. Y. Kei and S. S. Chow, "SHAFT: Secure, handy, accurate, and fast transformer inference," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2025.
[36] Y. Ding et al., "East: Efficient and accurate secure inference framework for transformer," *IEEE Trans. Services Comput.*, vol. 18, no. 4, pp. 2038–2046, Jul./Aug. 2025.
[37] K. Cheng et al., "Mosformer: Maliciously secure three-party inference framework for large transformers," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2025, pp. 4124–4138.
[38] H. Huang and Y. Wang, "SecBERT: Privacy-preserving pre-training based neural network inference system," *Neural Netw.*, vol. 172, 2024, Art. no. 106135.
[39] J. Luo et al., "SecFormer: Fast and accurate privacy-preserving inference for transformer models via SMPC," in *Proc. Findings Assoc. Comput. Linguistics ACL*, 2024, pp. 13333–13348.
[40] M. Zheng, Q. Lou, and L. Jiang, "Primer: Fast private transformer inference on encrypted data," in *Proc. 60th ACM/IEEE Des. Automat. Conf.*, 2023, pp. 1–6.
[41] T. Xu et al., "Breaking the layer barrier: Remodeling private transformer inference with hybrid {CKKS } and { MPC}," in *Proc. 34th USENIX Secur. Symp.*, 2025, pp. 2653–2672.
[42] J. Fan and F. Vercauteren, "Somewhat practical fully homomorphic encryption," *IACR Cryptol. ePrint Arch.*, vol. 2012, p. 144, 2012.
[43] Y. Lindell, "How to simulate it–A tutorial on the simulation proof technique," *Tut. Found. Cryptogr.: Dedicated to Oded Goldreich*, pp. 277–346, 2017.
[44] Y. Chen, Z. Tang, T. Lu, B. Zhang, Z. Shi, and Z. Wang, "Accelerating private large transformers inference through fine-grained collaborative computation," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 7482–7497, 2025.
[45] R. Lehmkuhl, P. Mishra, A. Srinivasan, and R. A. Popa, "Muse: Secure inference resilient to malicious clients," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 2201–2218.
[46] N. Chandran, D. Gupta, S. L. B. Obbattu, and A. Shah, "{SIMC }:{ ML} inference secure against malicious clients at {Semi-Honest}," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 1361–1378.
[47] M. Keller, "MP-SPDZ: A versatile framework for multi-party computation," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2020, pp. 1575–1590.
[48] B. Yuan, S. Yang, Y. Zhang, N. Ding, D. Gu, and S.-F. Sun, "{MD-ML }: Super fast { Privacy-Preserving} machine learning for malicious security with a dishonest majority," in *Proc. 33 rd USENIX Secur. Symp.*, 2024, pp. 2227–2244.
[49] Y. Zhang et al., "MD-SONIC: Maliciously-secure outsourcing neural network inference with reduced online communication," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 3534–3549, 2025.



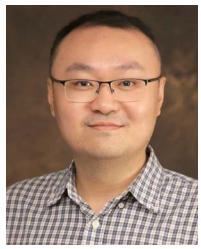
Mingshun Luo received the MS degree in cyberspace security from North China University of Technology, Beijing, China, in 2024. He is currently working toward the PhD degree with North China Electric Power University, Beijing. His research interests include privacy-preserving machine learning and privacy computing.



Haolei He received the BE degree from North China Electric Power University, Beijing, China, in 2024. He is now studying artificial intelligence with North China Electric Power University, Beijing. His research interests include privacy-preserving machine learning and privacy computing.



Wenti Yang received the PhD degree in information security from the North China Electric Power University, Beijing, China, in 2024. She is currently an assistant researcher with the Department of New Networks, Pengcheng Laboratory (PCL), Shenzhen, Guangdong, China. Her research focuses on privacy-preserving machine learning and security in computing power networks.



Meng Li (Senior Member, IEEE) is a professor with the School of Computer Science and Information Engineering, Hefei University of Technology (HFUT), China. He was a post-doc researcher with the Department of Mathematics and HIT Center, University of Padua, Italy, where he is with the Security and PRIvacy Through Zeal (SPRITZ) research group led by Prof. Mauro Conti (IEEE Fellow). His research interests include data sharing, IoV, security, privacy, applied cryptography, blockchain, TEE. In this area, he has published 154 papers in topmost journals and conferences, including T-IFS, TDSC, ToN, TMC, JSAC, TKDE, TODS, TPDS, TSE, TSC, COMST, IEEE S&P, USENIX Security, MobiCom, and INFOCOM.



Longfei Wu (Member, IEEE) received the BE degree from the Beijing University of Posts and Telecommunications, in 2012, and the PhD degree in computer and information sciences from Temple University, in 2017. He is currently an assistant professor with the Department of Mathematics and Computer Science, Fayetteville State University. His research interests are the security and privacy of networked systems and modern computing devices, including mobile devices, IoT, implantable medical devices, and wireless networks.



Zijian Zhang (Senior Member, IEEE) received the PhD degree from the School of Computer Science and Technology, Beijing Institute of Technology. He was a visiting scholar with the Computer Science and Engineering Department, State University of New York at Buffalo, in 2015. He is currently a professor with the School of Cyberspace Science and Technology, Beijing Institute of Technology. His research interests include design of authentication and key agreement protocol and analysis of entity behavior and preference.



Zhitao Guan (Member, IEEE) received the BEng and PhD degrees in computer application from the Beijing Institute of Technology, China, in 2002 and 2008, respectively. He is currently a professor with the School of Control and Computer Engineering, North China Electric Power University. His current research focuses on blockchain, AI security, privacy computing. He has authored more than 100 peer-reviewed journal and conference papers in these areas.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on November 08,2026 at 04:12:15 UTC from IEEE Xplore. Restrictions apply.